ESP32-C3 GPS Tracker v3.3 蓝牙模拟器技术对接文档

版本: v1.0.0
日期: 2026-03-14
适用对象: 前端/客户端开发工程师
用途: 本地硬件模拟器部署及蓝牙通信协议对接

---

1. 快速开始（部署指南）

1.1 环境要求

- **操作系统**: macOS 10.14+ / Windows 10+ / Linux (Ubuntu 18.04+)
- **Node.js**: v14.0.0 或更高版本
- **蓝牙适配器**: 内置或外置 USB 蓝牙 4.0+（BLE 支持）
- **网络**: 本地回环（localhost）或局域网访问

1.2 安装步骤

解压项目文件`esp_gps_blue.zip`文件，并进入该目录esp_gps_blue下，执行下面命令

```
# 1. 安装依赖
npm install

# 2. 启动服务
npm start
```

1.3 验证运行

启动成功后，控制台应显示：

```
========================================
  ESP32-C3 模拟器已启动
========================================
[控制面板] http://localhost:4444
[BLE设备]  ESP32-C3-Tracker

请用手机访问您的云端页面，搜索并连接 "ESP32-C3-Tracker"
```

访问控制面板:
浏览器打开 `http://localhost:4444`，可实时调节：

- GPS 坐标（纬度/经度）
- 卫星状态（GPS/北斗可见数、信号强度 SNR）
- 电池电量、充电状态
- 加速度计 XYZ 轴数据
- NFC 卡片 UID 模拟
- 等等

1.4 通过网页进行测试

拿出另一台设备，进入页面https://paxiusiai.cn/

在里面可以和上述的模拟设备进行匹配和控制/查看

2. 蓝牙通信协议详解

2.1 服务与特征值定义

项目	UUID	属性	说明
主服务	`12345678-1234-1234-1234-123456789abc`	Primary Service	ESP32-C3 主服务
TX特征	`12345678-1234-1234-1234-123456789abd`	Notify	设备 → 手机（传感器数据）
RX特征	`12345678-1234-1234-1234-123456789abe`	Write	手机 → 设备（控制指令）

设备名称: `ESP32-C3-Tracker`
广播间隔: 100ms（可连接状态）

2.2 上行数据格式（设备 → 手机）

模拟器通过 **Notify** 特征主动推送以下三类数据：

2.2.1 传感器数据包（周期：500ms）

```
{
  "a": [0.12, -0.05, 0.98],        // 加速度计 [X, Y, Z] 单位: g
  "b": [3.85, 85, 0],              // 电池 [电压(V), 电量(%), 充电状态(1/0)]
  "g": [1, 31.24512, 121.48235, 15, 8, 0.5, 1.2],
  // GPS数组索引定义:
  // [0] 定位有效标志 (1=有效, 0=无效)
  // [1] 纬度 (度, 保留5位小数)
  // [2] 经度 (度, 保留5位小数)
  // [3] 海拔 (米, 整数)
  // [4] 参与定位卫星数 (整数)
  // [5] 速度 (km/h, 保留1位小数)
  // [6] HDOP 水平精度因子 (保留1位小数)
  "t": "14:32:08",                 // UTC时间 HH:MM:SS
  "d": "13/03/26"                  // 日期 DD/MM/YY
}
```

解析示例（JavaScript）：

```javascript
const data = JSON.parse(receivedValue);
const [valid, lat, lon, alt, sats, speed, hdop] = data.g;

if (valid === 1 && lat !== 0) {
  console.log(`定位成功: ${lat}°, ${lon}°`);
  console.log(`精度: HDOP ${hdop}, 卫星数: ${sats}`);
}
```

2.2.2 GPS 诊断数据包（周期：3000ms 或请求触发）

```
{
  "gps_diag": {
    "ant": "OK",                    // 天线状态枚举: "OK" | "OPEN" | "SHORT" |
"UNKNOWN"
    "fix": 3,                       // 定位类型: 1=无定位, 2=2D定位, 3=3D定位
    "qual": 2,                      // 定位质量: 0=无效, 1=标准GPS, 2=差分GPS(DGPS)
    "sat_use": 8,                   // 实际参与定位解算的卫星数
    "sat_gp": 12,                   // GPS系统可见卫星总数
    "sat_bd": 6,                    // 北斗系统可见卫星总数
    "sat_tot": 18,                  // 总可见卫星数 = sat_gp + sat_bd
    "hdop": 1.2,                    // 水平 dilution of precision
    "snr": [                        // 卫星信号强度数组（最多8颗）
      {"p": 3, "s": 45, "t": "G"},  // p=PRN(伪随机码), s=SNR(dB-Hz), t=系统
(G=GPS, B=北斗)
      {"p": 12, "s": 38, "t": "G"},
      {"p": 201, "s": 42, "t": "B"},
      {"p": 204, "s": 35, "t": "B"}
    ]
  }
}
```

关键字段说明:

- `ant` **(Antenna)**: 硬件天线状态检测，"OPEN"表示天线断开，"SHORT"表示短路
- `fix`: 定位维度，3表示三维定位（含海拔），2表示二维（仅平面）
- `snr`: 信噪比，单位 dB-Hz。>40为强信号，25-40为中等，<25为弱信号
- `sat_tot`: 自动计算字段，始终等于 `sat_gp + sat_bd`

2.2.3 NFC 刷卡事件（触发式，非周期）

当检测到 NFC 卡片时立即发送（模拟器每 200ms 扫描一次）：

```
{
  "nfc": {
    "uid": "A1B2C3D4",              // 卡片 UID, 4字节十六进制大写字符串
    "sak": 8,                       // Select Acknowledge, 卡片选择应答字节（十进制）
    "type": "MIFARE_1K"             // 卡片类型字符串
  }
}
```

SAK 与类型对应表: | SAK值 | 类型标识 | 说明 | |-------|----------|------| | 0x04 (4) | MIFARE_UL | MIFARE Ultralight | | 0x08 (8) | MIFARE_1K | MIFARE Classic 1K | | 0x09 (9) | MIFARE_MINI |

MIFARE Mini | | 0x18 (24) | MIFARE_4K | MIFARE Classic 4K | | 0x20 (32) | MIFARE_PLUS | MIFARE Plus |

2.3 下行指令格式（手机 → 设备）

通过 **Write** 特征发送 UTF-8 字符串指令：

2.3.1 震动控制指令

格式: `VIB:<模式>:[参数1]:[参数2]:[参数3]`

指令	功能	持续时间	响应
`VIB:0`	停止震动	立即	LED恢复蓝/绿
`VIB:1`	短震	100ms	LED变红
`VIB:2`	长震	500ms	LED变红
`VIB:3`	双击	100ms×2 (间隔100ms)	LED变红
`VIB:4`	三击	100ms×3 (间隔100ms)	LED变红
`VIB:5`	SOS模式	三短(100)+三长(300)+三短(100)，共2500ms	LED变红
`VIB:6`	心跳模式	模拟心跳：50ms+100ms+100ms+400ms，共1000ms	LED变红
`VIB:7:200:200:3`	自定义	格式: `VIB:7:开启时间(ms):关闭时间(ms):重复次数`	LED变红

自定义震动参数范围:

- 开启时间: 10-2000ms
- 关闭时间: 10-2000ms
- 重复次数: 1-20次

Web Bluetooth 发送示例:

```
const encoder = new TextEncoder();
const command = 'VIB:3'; // 双击
await rxCharacteristic.writeValue(encoder.encode(command));
```

2.3.2 NFC 控制指令

指令	功能	响应
`NFC:SCAN`	强制立即扫描 NFC	若卡片存在，立即发送 NFC JSON 事件

注意: 正常情况下模拟器每 200ms 自动扫描，此指令用于手动触发。

2.3.3 GPS 诊断指令

指令	功能	响应
`GPS:DIAG`	立即发送 GPS 诊断数据包	返回 `gps_diag` JSON，不等待 3000ms 周期

---

3. 通信时序图

```
时间轴 (ms)
0        500       1000      1500      2000      2500      3000
|         |         |         |         |         |         |
├传感器─┼传感器─┼传感器─┼传感器─┼传感器─┼传感器─┤  ← 每500ms自动发送
|         |         |         |         |         |         |
|         |    [NFC扫描每200ms，检测到新卡时立即推送nfc事件]    |
|         |         |         |         |         |         |
├──────────────────────────────────────┼GPS诊断─┤ ← 每3000ms自动发送
|                                       |         |
|         [收到 VIB:1 立即执行]       |         |
|         [收到 GPS:DIAG 立即回复]     |         |
|         [收到 NFC:SCAN 立即扫描]     |         |
```

连接生命周期:

1. **广播阶段**: 设备发出 `ESP32-C3-Tracker` 广播（绿色 LED）
2. **连接阶段**: 手机连接并订阅 Notify（蓝色 LED）
3. **数据传输**: 自动开始 500ms/3000ms 周期推送
4. **指令交互**: 可随时通过 Write 发送控制指令
5. **断开阶段**: 停止广播，恢复绿色 LED 待机

4. 完整对接示例代码

4.1 Web Bluetooth API 连接示例

```javascript
// UUID 定义（必须与模拟器一致）
const SERVICE_UUID = '12345678-1234-1234-1234-123456789abc';
const CHAR_TX_UUID = '12345678-1234-1234-1234-123456789abd';
const CHAR_RX_UUID = '12345678-1234-1234-1234-123456789abe';

let device, server, txChar, rxChar;

async function connectDevice() {
  try {
    // 1. 请求设备（用户选择 ESP32-C3-Tracker）
    device = await navigator.bluetooth.requestDevice({
      filters: [{ name: 'ESP32-C3-Tracker' }],
      optionalServices: [SERVICE_UUID]
    });

    // 2. 连接 GATT
    server = await device.gatt.connect();
    const service = await server.getPrimaryService(SERVICE_UUID);

    // 3. 获取特征值
    txChar = await service.getCharacteristic(CHAR_TX_UUID);
    rxChar = await service.getCharacteristic(CHAR_RX_UUID);

    // 4. 订阅数据接收
    await txChar.startNotifications();
    txChar.addEventListener('characteristicvaluechanged', handleData);

    console.log('✅ 连接成功');
  } catch (error) {
    console.error('❌ 连接失败:', error);
  }
}

// 数据接收处理
function handleData(event) {
  const value = new TextDecoder().decode(event.target.value);

  try {
    const data = JSON.parse(value);

    // 处理传感器数据
    if (data.a && data.b && data.g) {
      const [ax, ay, az] = data.a;
      const [voltage, percent, charging] = data.b;
      const [valid, lat, lon, alt, sats, speed, hdop] = data.g;

      console.log(`传感器: 电量${percent}%, 坐标[${lat}, ${lon}]`);
```

```
    }

    // 处理 GPS 诊断
    if (data.gps_diag) {
      const diag = data.gps_diag;
      console.log(`GPS诊断: 天线${diag.ant}, 定位${diag.fix}D, 卫星
${diag.sat_use}/${diag.sat_tot}`);

      // SNR 信号强度
      diag.snr.forEach(sat => {
        console.log(`  卫星 ${sat.t}${sat.p}: ${sat.s} dB-Hz`);
      });
    }

    // 处理 NFC
    if (data.nfc) {
      console.log(`NFC刷卡: UID=${data.nfc.uid}, 类型=${data.nfc.type}`);
    }

  } catch (e) {
    console.error('JSON解析失败:', value);
  }
}

// 发送震动指令
async function vibrate(mode) {
  if (!rxChar) return;
  const cmd = `VIB:${mode}`;
  await rxChar.writeValue(new TextEncoder().encode(cmd));
}

// 请求立即 GPS 诊断
async function requestGpsDiag() {
  if (!rxChar) return;
  await rxChar.writeValue(new TextEncoder().encode('GPS:DIAG'));
}
```

4.2 React/Vue 集成建议

```
// 推荐封装为 Hook/Composable
export function useEsp32Ble() {
  const [connected, setConnected] = useState(false);
  const [sensorData, setSensorData] = useState(null);
  const [gpsDiag, setGpsDiag] = useState(null);
  const [nfcEvent, setNfcEvent] = useState(null);

  // 实现 connect/disconnect/sendCommand 等方法

  return {
    connected,
    sensorData,     // 实时传感器数据
    gpsDiag,        // GPS诊断信息（每3秒更新）
    nfcEvent,       // 最新NFC刷卡事件
    vibrate,        // 震动控制函数
    requestDiag     // 手动请求诊断
  };
}
```

---

5. 故障排查

5.1 无法发现设备

- **检查**: 本地电脑蓝牙是否开启?
- **检查**: 模拟器是否已启动（控制台显示"广播"）？
- **检查**: 浏览器是否支持 Web Bluetooth（Chrome/Edge）？

5.2 连接成功但无数据

- **检查**: 是否调用了 `startNotifications()` 订阅 TX 特征?
- **检查**: 控制面板是否设置了 `ant` 为 "OK" 而非 "UNKNOWN"?
- **检查**: 浏览器控制台是否有 JSON 解析错误?

5.3 指令发送无响应

- **检查**: 是否在 `rxChar` 上调用 `writeValue`?
- **格式**: 确保指令为 UTF-8 字符串，如 `"VIB:1"` 而非对象
- **权限**: Web Bluetooth 要求用户手势触发（如点击按钮）

---

6. 版本历史

版本	日期	变更内容
v1.0.0	2026-03-14	初始版本，支持传感器/GPS诊断/NFC/震动控制

技术支持: 硬件团队
文档维护: 硬件团队
相关代码: `esp_gps_blue.zip` (详见项目目录)